

Git for Data Engineers

Just don't share code with your teammates via Google Drive like I did.



VU TRINH

JUL 08, 2025 • PAID



60



2



3

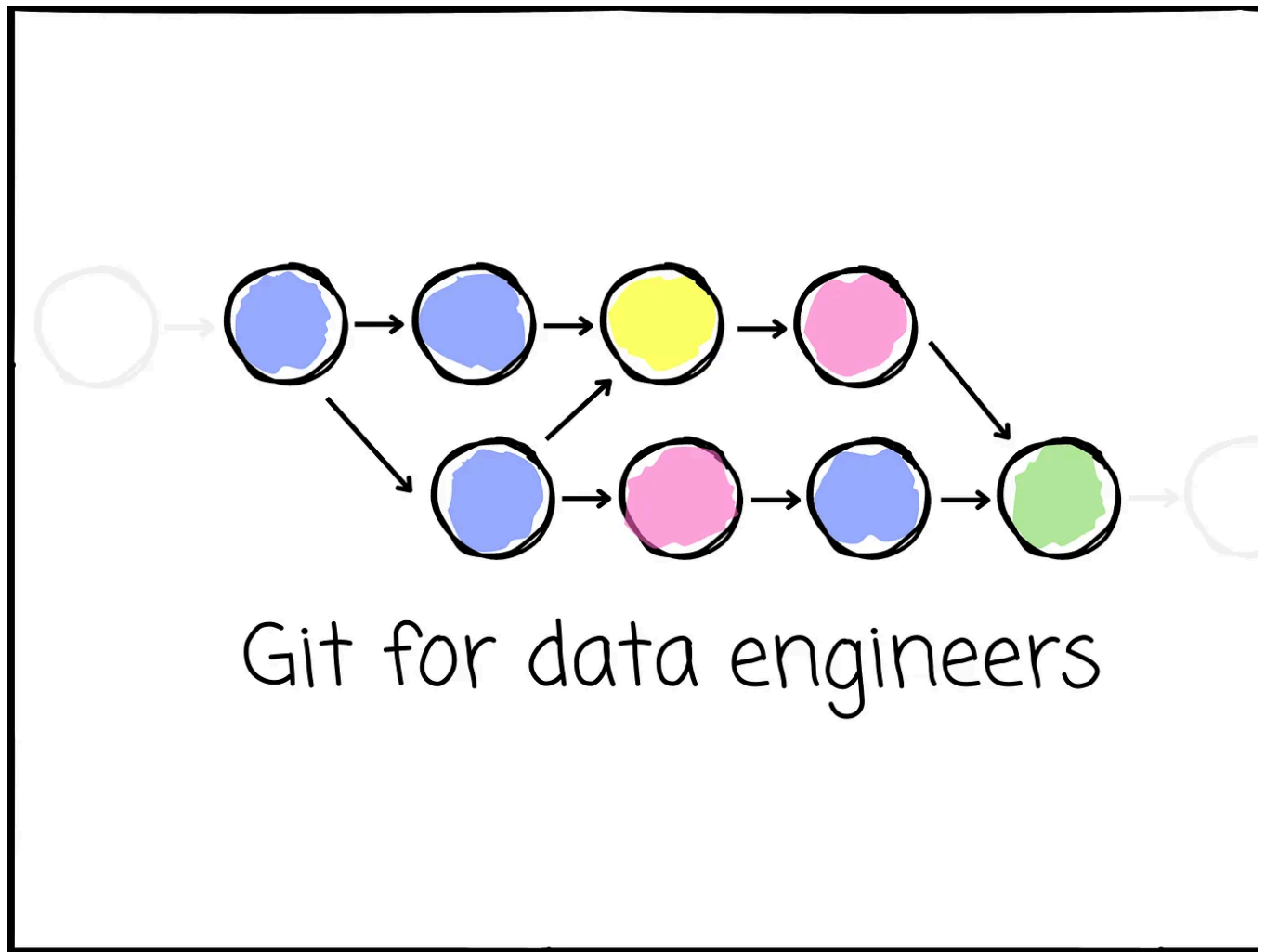
SI

I will publish a paid article every Tuesday. I wrote these with one goal in mind: to offer readers, whether they are feeling overwhelmed when beginning the journey or seeking a deeper understanding of the field, 15 minutes of practical lessons and insights on nearly everything related to data engineering.

*I invite you to join the club with a **50% discount on the yearly package**. Let's not be stuck in data engineering together.*

Subscribe





Intro

During my final year at university, I collaborated with a friend on a capstone project: a simple Python application. We needed a way to share what we've done. Guess what? We shared Python scripts via Google Drive.

Unsurprisingly, this caused us a lot of trouble managing the code, as it lacked version control and the ability to isolate each other's work. If we had chosen Git in the first place, we wouldn't be suffering like that.

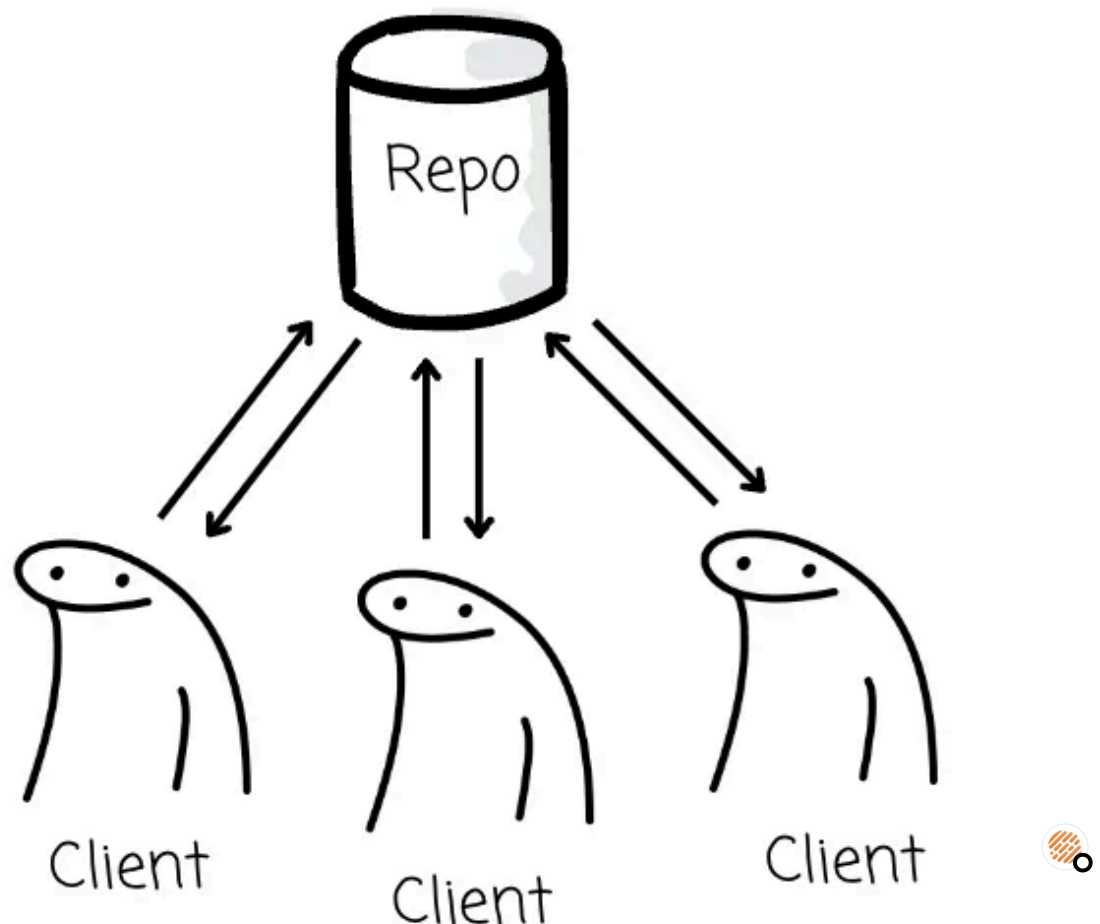
Seven years passed, and Git is now a part of my job. But here is my confession: I don't truly understand how it works behind the scenes. I decided to sit down and relearn this. This time, it's not only about some Git commands, but also about what happens under the hood.

Centralized vs distributed version control system

Before learning Git, it would be helpful to understand the differences between centralized and distributed version control systems.

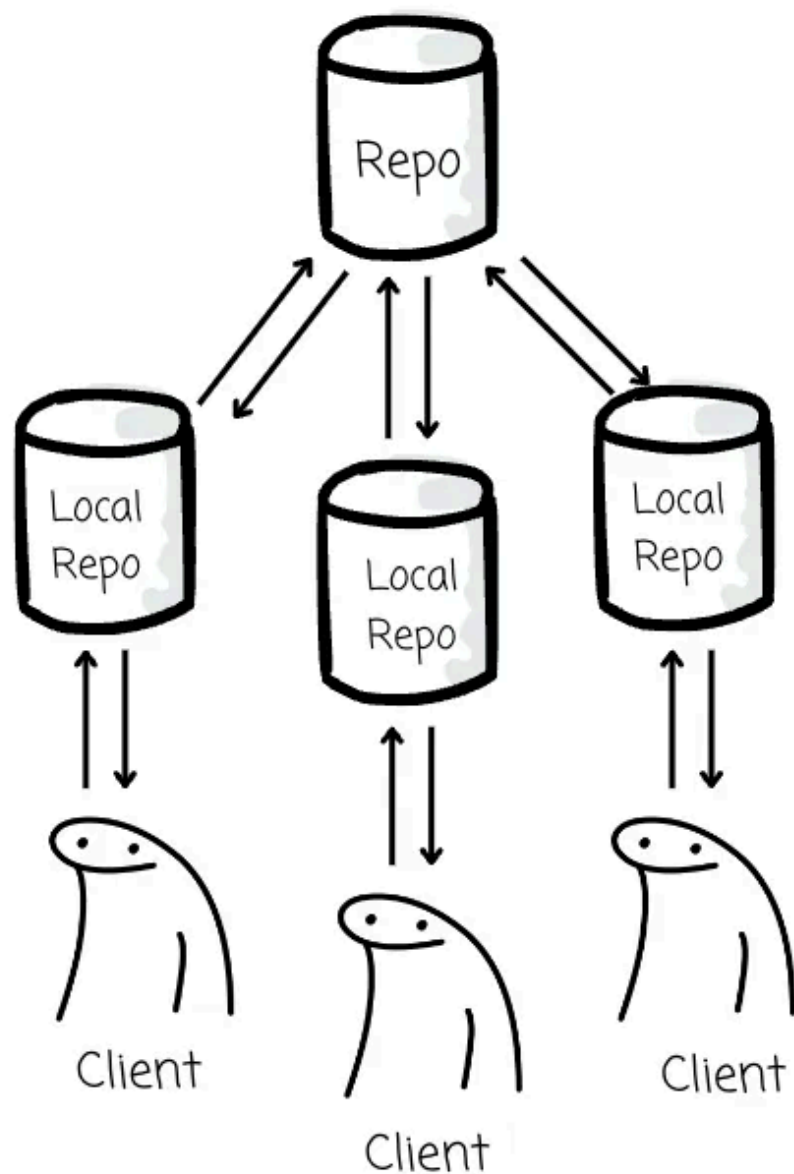
A version control system (VCS) tracks changes to files over time, facilitating team collaboration and coordination. Sharing via a shared Drive will soon become a nightmare, as we can only overwrite the files without any version tracking. Naming file differently with a new version could work, but it is highly error-prone.

People developed a Centralized Version Control System (CVCS) to deal with this problem. The CVCS has a server that manages all the versioned files. The client can create local copies of the files they want to work with. When done, they update the changes to the central server.



However, it has disadvantages. If the server fails, the whole codebase will be gone. Additionally, most operations in a CVCS require a constant connection to the central server. This is problematic for developers working with an unreliable internet connection.

That's why a distributed version control system (DVCS) was developed.



It provides every developer with a complete copy of the entire repository, including the whole history. This eliminates the single point of failure inherent in CVCS. Operations like commits and branching are lightning-fast and entirely local (as each client keeps a local repository), solving the performance and offline work limitations of CVCS.

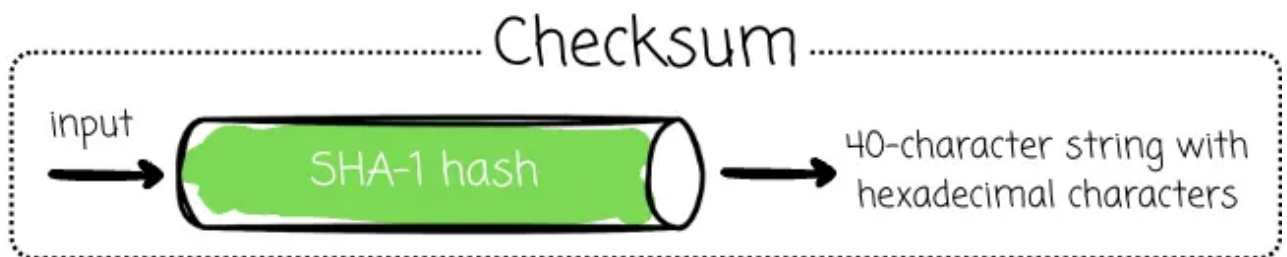
Git is an **open-source** DVCS developed by the Linux development community in 2

Git overview

Compared to other VCS, Git sees the data as snapshots rather than delta changes. provides powerful branching capability by leveraging lightweight pointers. We will learn more about commits and branches in the following sections.



To ensure integrity, Git checksums everything. This helps Git check file changes efficiently. Git uses SHA-1 hash for the checksum; the hash function inputs are the file/directory contents, and the result is a 40-character string with hexadecimal characters (0–9 and a–f)



Due to the nature of a DVCS, most Git operations can be performed locally, a  client will have a local repo on your computer. Users persist the changes locally. Only when they need to collaborate do they interact with the remote repo via the internet.

We will learn about the local and remote repo in the next section.

The local repo

In local, Git manages your files in 3 sections:

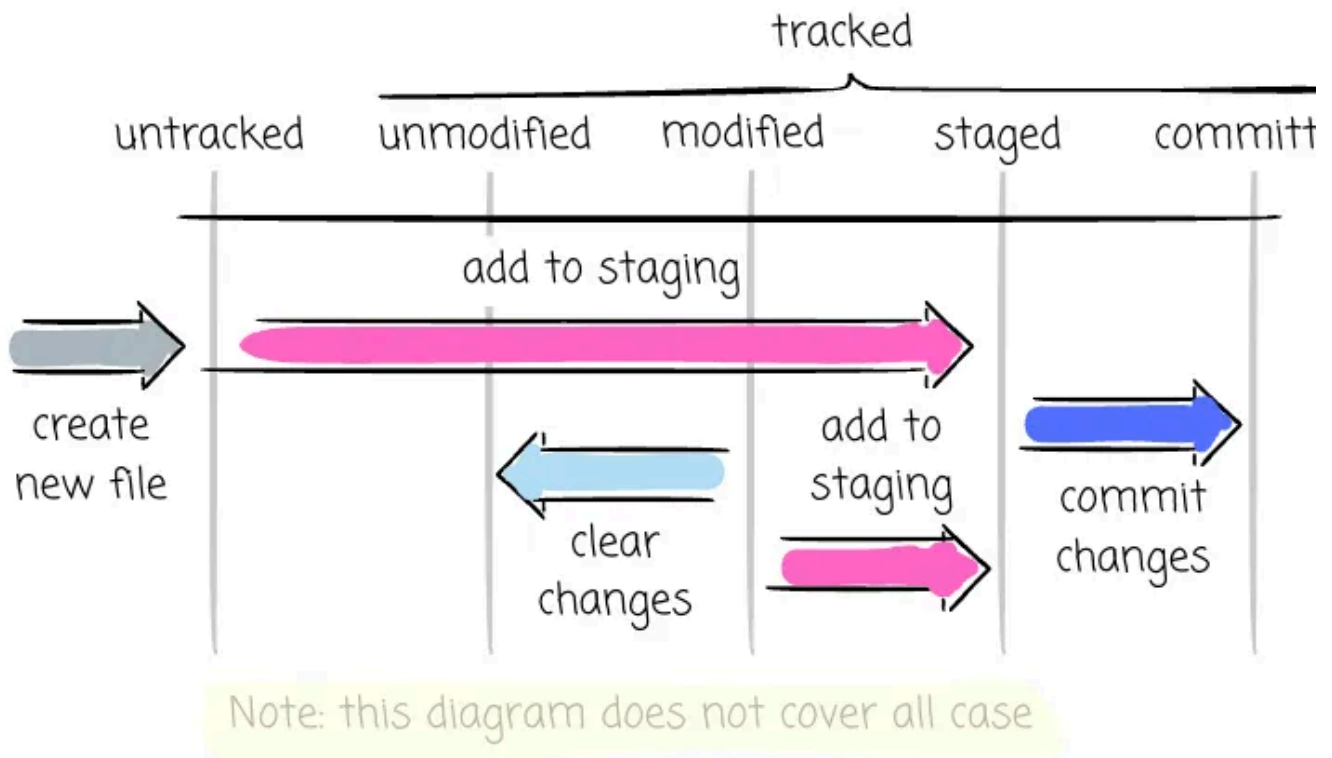


- **Working directory:** This is where we can see and edit the actual files.
- **Staging area:** It is a file inside a `.git` directory that holds the information of what will be included in the next commit.
- **The local repository (.git dir):** This is where Git stores the history of your project as a series of commit objects locally.

A typical workflow looks like this: you make changes to files, then you choose what to commit by staging them; when everything is ready, you commit your changes to the local repository.

In the working directory, a file could be in one of the two states:



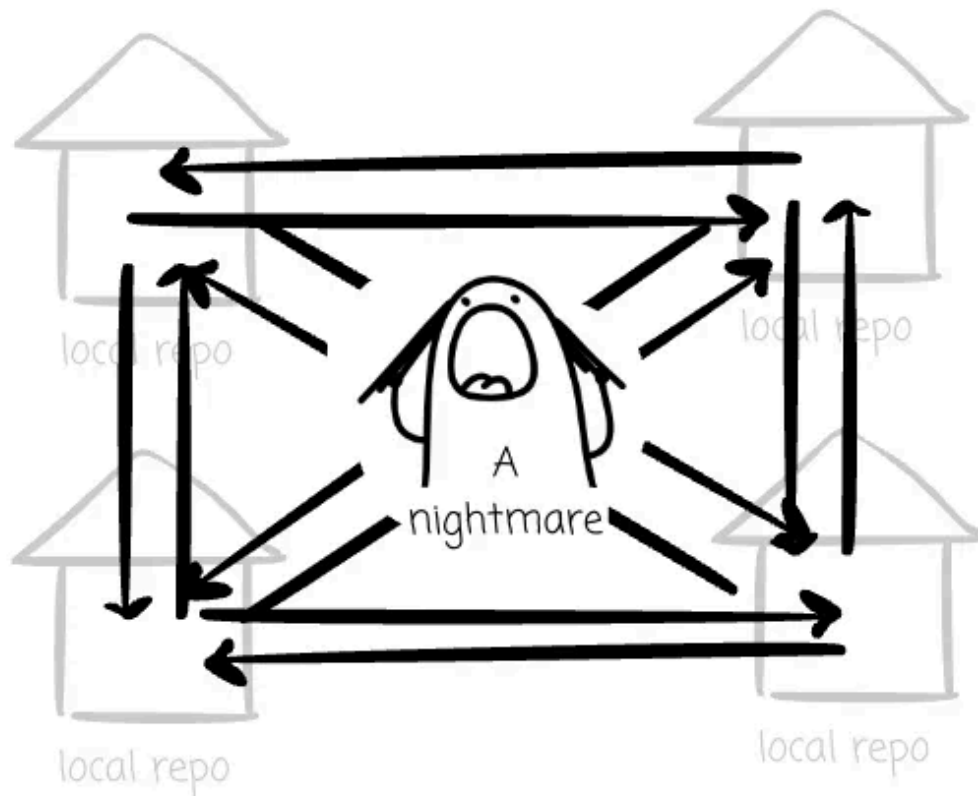


- **Untracked:** These are files in your working directory that were not in the last commit and are not in the staging area. Git is not familiar with these files. This is the state of any brand-new file.
- **Tracked:** These are files that Git knows about. They were part of the last commit and Git is actively tracking them. Tracked files can exist in one of the following sub-states: **unmodified**, **modified**, **staged**, or **committed**.

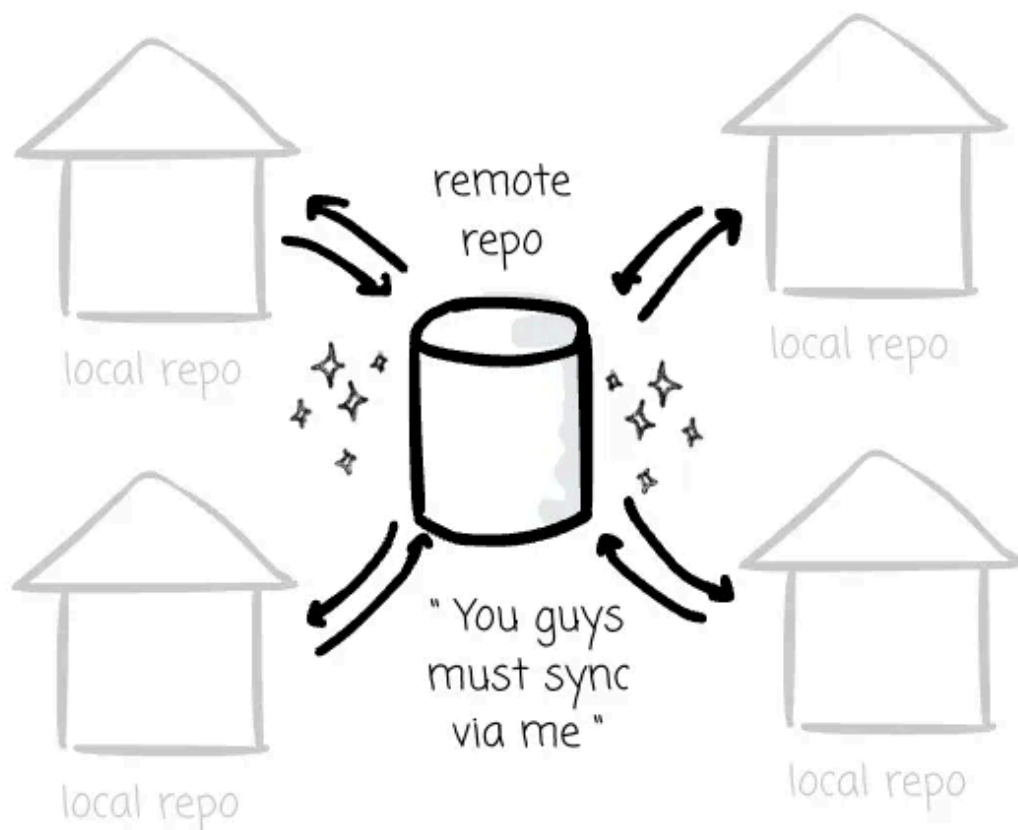
Committed means your changes are persisted in the repository. **Staged** means your files are included in the stage area. **Modified** means Git detects that your files have changed compared to the latest commit, but they are not staged or committed. **Unmodified** means your files remain unchanged compared to the last commit.

The remote repo

While it's true that every developer has a complete local copy of the repository in a DVCS, it would be a mess if they connected directly to each other's laptops to share the updates.



With a remote repository, everyone synchronizes their work with this central hub.



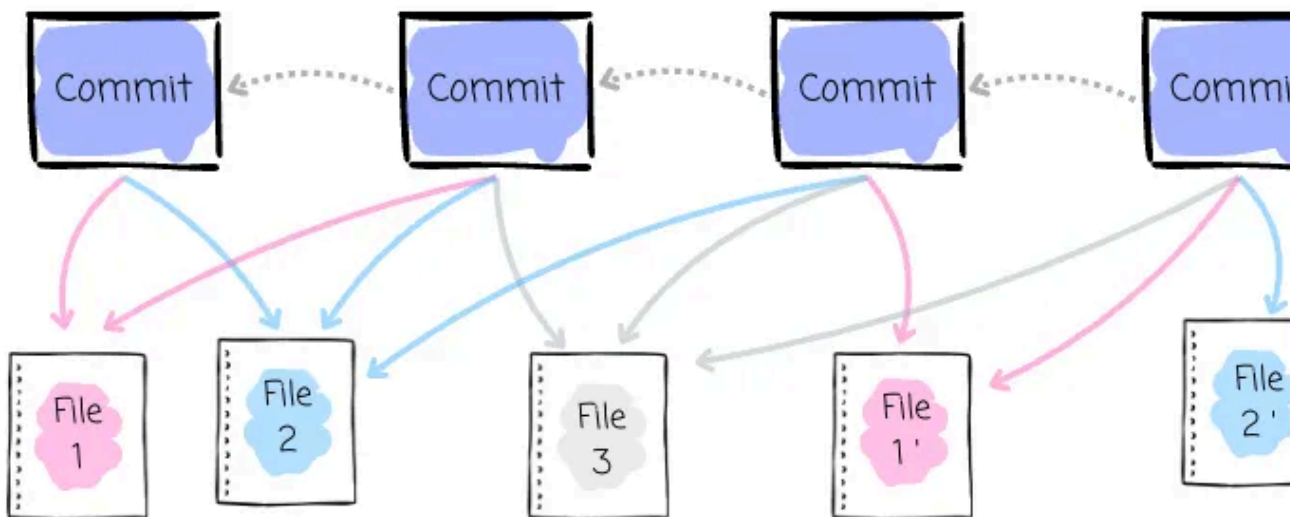
The three sections above are on our local laptop. The remote repository is another version of your project that is located "elsewhere"—on platforms like GitHub, Git or Bitbucket.

To share your changes, you can “push” your committed changes from your local repository to the remote repository. In the reverse direction, you can retrieve any changes (commits/branches) from the remote repository into your local repository. A typical workflow now includes one step: you interact with the remote repository.

Commit

We heard that changes will be committed to the repository. So, what is a commit?

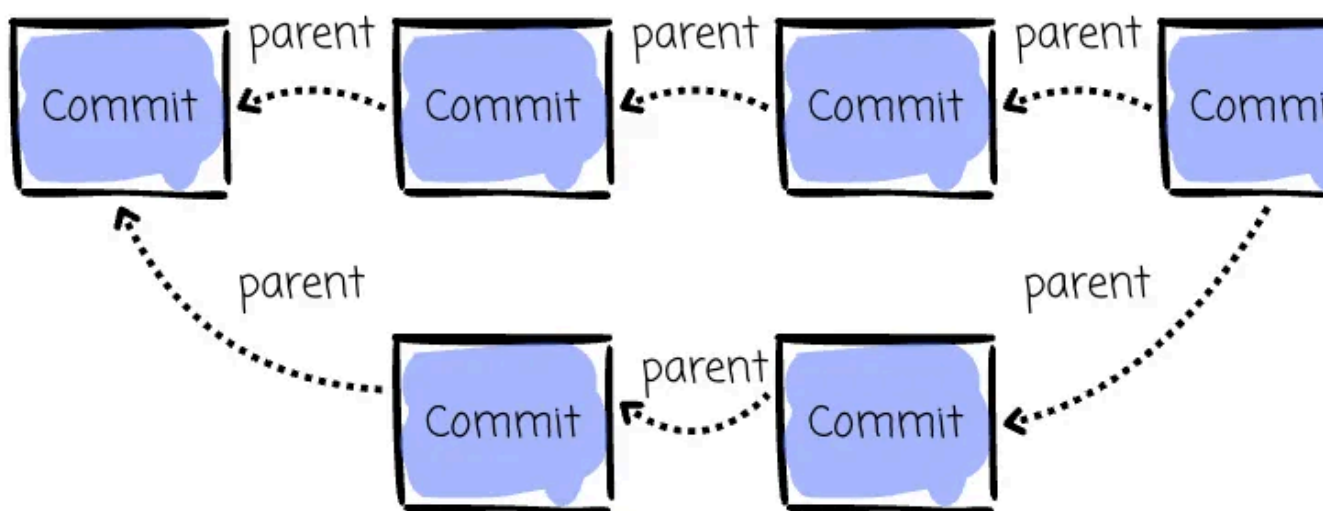
A commit is a snapshot of the entire project at a specific point in time. It's not just a record of what changed; it's a complete picture of every file and folder in the repository exactly as they were when we made the commit. A commit is identified by its SHA-1 hashed identifier.



This snapshot is created from the files in the staging area. The commit actions record new versions for these staged files in the local repository's history. If files don't have changes, Git only needs to store the links to the files in the previous commits.

Besides the snapshot, a commit also has the metadata:

- The name and email of the one who made the commit.
- A unique identifier that is calculated from the SHA-1 hash.
- A timestamp
- A commit message
- A pointer to the parent. A regular commit has one parent, and a merge commit could have more than one parent.

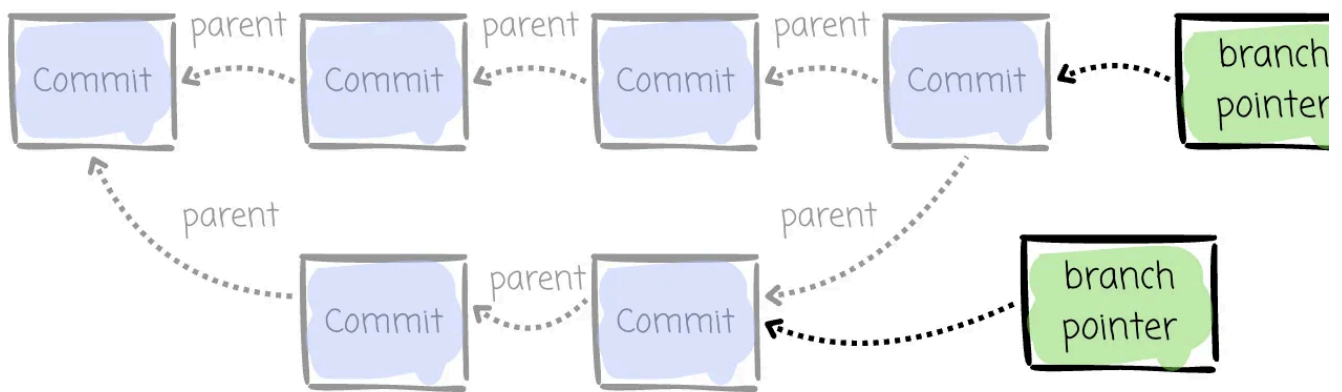


Branch

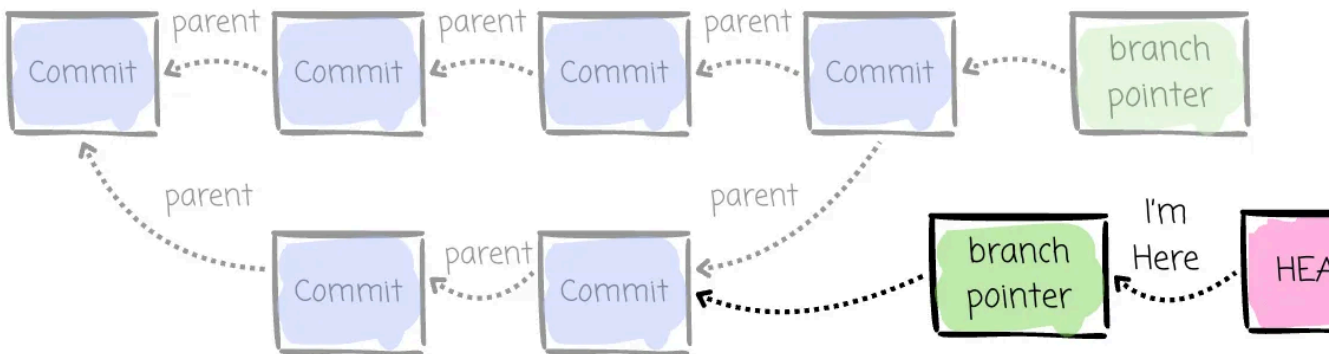
Git branches allow for parallel development.

As mentioned, a commit has a pointer to its parent commit(s). A Git branch is simply a movable pointer to a commit. When creating the repository for the first time, a default branch will be created.

Creating a new branch from an existing branch only creates a new pointer that points to the latest commit of the existing branch. As a result, the branch creation process is extremely lightweight.



Git keeps a special pointer called **HEAD** (it points to the current branch pointer) to identify the current commit you're working on. Also, when we make changes or create new commits, Git uses **HEAD** to know where those new commits should be added.



Summary before moving on

We've learned that Git is a distributed version control system, where each client can have their repo on their laptop. Git tracks your file changes locally in three sections: the working directory, the staging area, and the local repository.

A commit is a snapshot of the entire project at a specific point in time. A commit points to its parent commit. A branch is simply a movable pointer to a commit. We can share your work or get updates from others by interacting with a remote repository.

Next, we will delve into some Git commands to gain a deeper understanding of what happens behind the scenes. Imagine a scenario where a team is working on a project.

called Boring1.

Git clone

This post is for paid subscribers

[Subscribe](#)

Already a paid subscriber? [Sign in](#)

© 2025 Vu Trinh • [Privacy](#) • [Terms](#) • [Collection notice](#)
[Substack](#) is the home for great culture

